Task 1B: NoSQL Design Brief (MongoDB)

Scenario (from Lab 1B): a store needs a `products` catalogue (read constantly, rarely changed) and a `user_events` log of views, carts and purchases (written constantly, read by analytics).

Document structure: a deliberate mix

I designed around access patterns rather than normalisation.

`products` uses embedding. A product page needs price, stock, rating summary, supplier and tags in one read, so they live in one document. The rating is stored as a pre-computed summary (`{average, count}`) and only the three newest reviews are embedded. Full review history goes in a separate `reviews` collection referencing the `sku`, because it grows without limit and could push a popular product towards the 16 MB document cap.

`user_events` uses referencing with snapshots. Events store `user_id` and `product_sku` as references; embedding whole products would bloat millions of documents with stale copies. I do snapshot `price_at_event` and `category`: analytics needs the price the customer actually saw, and a local `category` lets the commonest aggregation (`$match` on category and time, then `$group`) skip `$lookup`. As a time-series collection with a TTL index, raw events expire after 13 months, which supports GDPR storage limitation.

Key indexes: `{category: 1, price: 1}` and multikey `{tags: 1}` on products; `{category: 1, timestamp: -1}` and `{user_id: 1, timestamp: -1}` on events.

MongoDB compared with PostgreSQL for this scenario

In PostgreSQL the same product page would need joins across products, ratings, suppliers and a tags link table, and category-specific attributes (screen size, fabric) would need nullable columns, EAV or JSONB. MongoDB returns it in one indexed read and lets each category carry its own fields. The trade-off is integrity: PostgreSQL enforces foreign keys and natural multi-row transactions, which is why my portfolio keeps orders relational (Task 1A) and uses MongoDB for catalogue and clickstream.

Where the CAP theorem fits

During a network partition a distributed database must choose consistency or availability. A MongoDB replica set has one primary, so with `writeConcern: "majority"` and primary reads it behaves as **CP**: a cut-off minority cannot accept writes. I would not apply one setting everywhere. Stock and purchase writes keep `majority`, because overselling is worse than a short error. Catalogue browsing can read from secondaries, accepting seconds-stale prices for lower latency, and event inserts can use `w: 1`, since losing a few clicks in a failover changes no analytics result.

Appendix: example documents (not part of the word count)

```
// products
{
```

```
  "_id": "ELEC-001",
  "name": "Wireless Noise-Cancelling Headphones",
  "category": "Electronics",
  "price": 4999.00, "currency": "INR", "stock_quantity": 142,
  "attributes": { "battery_hours": 30, "bluetooth": "5.3" },
  "ratings": { "average": 4.3, "count": 2871 },
  "recent_reviews": [ { "user_id": "usr_10021", "score": 5, "text": "Great ANC", "at":
"2024-03-21" } ],
  "supplier": { "name": "TechDistrib Pvt Ltd", "lead_time_days": 7 },
  "tags": ["wireless", "noise-cancelling", "bluetooth"],
  "is_active": true
}

// user_events (time-series collection: timeField = "timestamp", metaField = "meta")
{
  "timestamp": "2024-03-20T14:32:11Z",
  "meta": { "user_id": "usr_10021", "device": "mobile" },
  "event_type": "purchase",
  "product_sku": "ELEC-001",
  "category": "Electronics",
  "price_at_event": 4999.00,
  "quantity": 1
}
```